

Lab 8 – Computation of LEADING and TRAILING

Aim

To compute the LEADING and TRAILING sets of the given operator grammar.

Definition

- **LEADING(A)**: the set of terminals that can appear as the **first terminal** in strings derived from non-terminal A.
- **TRAILING(A)**: the set of terminals that can appear as the **last terminal** in strings derived from non-terminal A.

Algorithm

LEADING

1. Start the program.
2. For each production $A \rightarrow \alpha$:
 - If first symbol is terminal, include it in LEADING(A).
 - If first symbol is non-terminal, include LEADING of that non-terminal.
 - If second symbol is terminal after a non-terminal, include it.
3. Repeat until no new symbols are added.

TRAILING

1. For each production $A \rightarrow \alpha$:
 - If last symbol is terminal, include it in TRAILING(A).
 - If last symbol is non-terminal, include TRAILING of that non-terminal.
 - If previous symbol is terminal before a non-terminal, include it.
2. Repeat until no new symbols are added.
3. Stop.

C Program

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX 10

char prod[MAX][20];
int n;

char leading[50];
```

```

char trailing[50];
int lIndex, tIndex;

void add(char *set, int *index, char c) {
    for (int i = 0; i < *index; i++) {
        if (set[i] == c)
            return;
    }
    set[( *index )++] = c;
}

void LEADING(char c);
void TRAILING(char c);

void LEADING(char c) {
    for (int i = 0; i < n; i++) {
        if (prod[i][0] == c) {
            if (!isupper(prod[i][2])) {
                add(leading, &lIndex, prod[i][2]);
            } else {
                if (prod[i][3] != '\0' && !isupper(prod[i][3]))
                    add(leading, &lIndex, prod[i][3]);
                LEADING(prod[i][2]);
            }
        }
    }
}

void TRAILING(char c) {
    for (int i = 0; i < n; i++) {
        if (prod[i][0] == c) {
            int len = strlen(prod[i]);
            char last = prod[i][len - 1];

            if (!isupper(last)) {
                add(trailing, &tIndex, last);
            } else {
                if (len - 2 >= 2 && !isupper(prod[i][len - 2]))
                    add(trailing, &tIndex, prod[i][len - 2]);
                TRAILING(last);
            }
        }
    }
}

int main() {
    char symbol;

    printf("Enter number of productions: ");
    scanf("%d", &n);

```

```

printf("Enter productions (Example: E=E+T)\n");
for (int i = 0; i < n; i++)
    scanf("%s", prod[i]);

printf("Enter non-terminal: ");
scanf(" %c", &symbol);

lIndex = 0;
memset(leading, 0, sizeof(leading));
LEADING(symbol);

printf("\nLEADING(%c) = { ", symbol);
for (int i = 0; i < lIndex; i++)
    printf("%c ", leading[i]);
printf("}\n");

tIndex = 0;
memset(trailing, 0, sizeof(trailing));
TRAILING(symbol);

printf("TRAILING(%c) = { ", symbol);
for (int i = 0; i < tIndex; i++)
    printf("%c ", trailing[i]);
printf("}\n");

return 0;
}

```

Sample Input

```

Enter number of productions: 3
E=E+T
E=T
T=i
Enter non-terminal: E

```

Sample Output

```

LEADING(E) = { + i }
TRAILING(E) = { + i }

```

Result

Thus the LEADING and TRAILING sets were computed successfully for the given grammar.